SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO1: Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)

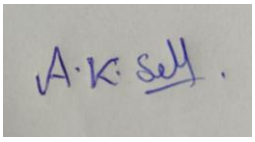
QUESTIONS: 5 **Total Marks: 100**

ANNEXURE A **Coding Challenge**

Code of Conduct:

I **A K Selva Prabhu (192421224)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



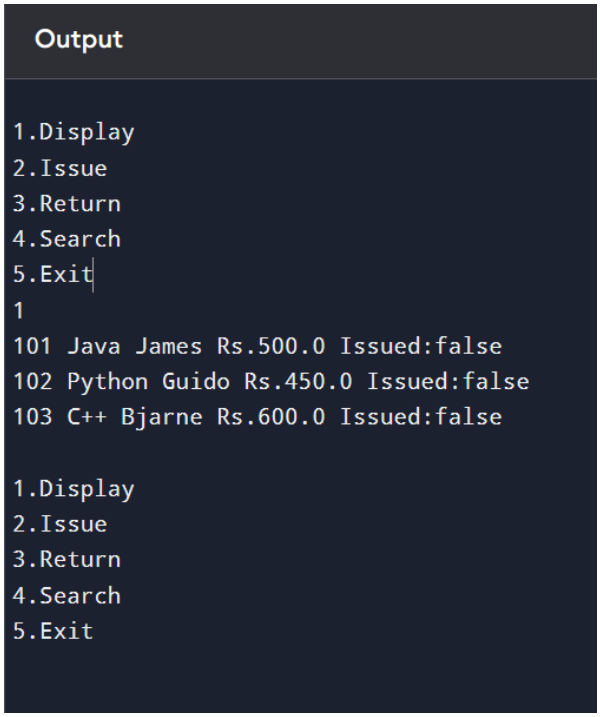
1. Library Book Management

```
import java.util.Scanner;

class Book {
    private int bookId;
    private String title;
    private String author;
    private double price;
    private boolean issued;

    Book(int id, String t, String a, double p) {
        bookId = id;
        title = t;
        author = a;
        price = p;
        issued = false;
    }

    public int getId() {
        return bookId;
    }

    public String getTitle() {
        return title;
    }

    public void issueBook() {
        if (!issued) {
            issued = true;
```

```
            System.out.println("Book Issued Successfully");
        } else {
            System.out.println("Book Already Issued");
        }
    }

    public void returnBook() {
        if (issued) {
            issued = false;
            System.out.println("Book Returned Successfully");
        } else {
            System.out.println("Book was not Issued");
        }
    }

    public void display() {
        System.out.println(bookId + " " + title + " " + author + " Rs." + price + " Issued:" +
issued);
    }
}

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Book books[] = new Book[3];

        books[0] = new Book(101, "Java", "James", 500);
```

```
books[1] = new Book(102, "Python", "Guido", 450);
books[2] = new Book(103, "C++", "Bjarne", 600);

int choice;

do {

    System.out.println("\n1.Display");
    System.out.println("2.Issue");
    System.out.println("3.Return");
    System.out.println("4.Search");
    System.out.println("5.Exit");

    choice = sc.nextInt();

    switch(choice){

        case 1:
            for(Book b:books)
                b.display();
            break;

        case 2:
            System.out.println("Enter Book ID");
            int id=sc.nextInt();
            for(Book b:books)
                if(b.getId()==id)
                    b.issueBook();
            break;
```

```
            case 3:
                System.out.println("Enter Book ID");
                id=sc.nextInt();
                for(Book b:books)
                    if(b.getId()==id)
                        b.returnBook();
                break;

            case 4:
                System.out.println("Enter Book ID");
                id=sc.nextInt();
                for(Book b:books)
                    if(b.getId()==id)
                        b.display();
                break;

        }

    }while(choice!=5);

  }
}
```



2. Employee Payroll (Inheritance)

```
class Employee {

    String name;

    Employee(String name) {
        this.name = name;
    }

    void calculateSalary() {
        System.out.println("Salary");
    }
}

class PermanentEmployee extends Employee {
```

```
    double basicSalary;

    PermanentEmployee(String name, double basicSalary) {
        super(name);
        this.basicSalary = basicSalary;
    }

    @Override
    void calculateSalary() {
        double salary = basicSalary + 5000;
        System.out.println("Permanent Employee");
        System.out.println("Name : " + name);
        System.out.println("Salary : " + salary);
    }
}

class ContractEmployee extends Employee {

    int hours;
    int rate;

    ContractEmployee(String name, int hours, int rate) {
        super(name);
        this.hours = hours;
        this.rate = rate;
    }

    @Override
    void calculateSalary() {
```

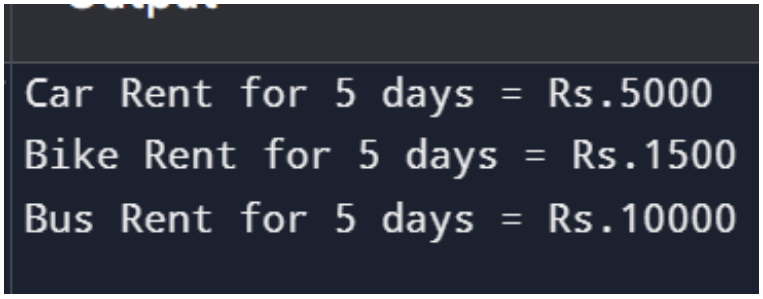
```
        double salary = hours * rate;
        System.out.println("Contract Employee");
        System.out.println("Name : " + name);
        System.out.println("Salary : " + salary);
    }
}

public class Main {

    public static void main(String[] args) {

        Employee e;

        e = new PermanentEmployee("Arun", 30000);
        e.calculateSalary();

        System.out.println();

        e = new ContractEmployee("Kumar", 120, 250);
        e.calculateSalary();
    }
}
```

```
Permanent Employee
Name : Arun
Salary : 35000.0

Contract Employee
Name : Kumar
Salary : 30000.0
```

3: Vehicle Rental System using Runtime Polymorphism

```
class Vehicle {

    void calculateRent(int days) {
        System.out.println("Vehicle Rent");
    }
}

class Car extends Vehicle {

    @Override
    void calculateRent(int days) {
        System.out.println("Car Rent for " + days + " days = Rs." + (days * 1000));
    }
}

class Bike extends Vehicle {

    @Override
    void calculateRent(int days) {
        System.out.println("Bike Rent for " + days + " days = Rs." + (days * 300));
    }
}

class Bus extends Vehicle {

    @Override
    void calculateRent(int days) {
        System.out.println("Bus Rent for " + days + " days = Rs." + (days * 2000));
```

```
    }
}

public class Main {

    public static void main(String[] args) {

        Vehicle[] vehicles = new Vehicle[3];

        vehicles[0] = new Car();
        vehicles[1] = new Bike();
        vehicles[2] = new Bus();

        int days = 5;

        for (Vehicle v : vehicles) {
            v.calculateRent(days);
        }
    }
}
```



4: Bank Account System using Data Hiding

```
class BankAccount {
```

```
// Private data members
private int accountNumber;
private String holderName;
private double balance;

// Constructor
BankAccount(int accountNumber, String holderName, double balance) {
    this.accountNumber = accountNumber;
    this.holderName = holderName;
    this.balance = balance;
}

// Deposit method
void deposit(double amount) {
    if (amount > 0) {
        balance = balance + amount;
        System.out.println("Amount Deposited Successfully.");
    } else {
        System.out.println("Invalid Deposit Amount.");
    }
}

// Withdraw method
void withdraw(double amount) {
    if (amount <= 0) {
        System.out.println("Invalid Withdrawal Amount.");
    } else if (amount > balance) {
        System.out.println("Insufficient Balance.");
    } else {
```

```
            balance = balance - amount;
            System.out.println("Amount Withdrawn Successfully.");
        }
    }

    // Display Account Details
    void displayBalance() {
        System.out.println("\nAccount Number : " + accountNumber);
        System.out.println("Account Holder : " + holderName);
        System.out.println("Available Balance : Rs." + balance);
    }
}

public class Main {

    public static void main(String[] args) {

        // Create Bank Account
        BankAccount account = new BankAccount(1001, "AK Selva", 5000);

        // Deposit Money
        account.deposit(2000);

        // Withdraw Money
        account.withdraw(1500);

        // Invalid Withdrawal
        account.withdraw(10000);
```

```
        // Invalid Deposit
        account.deposit(-500);

        // Display Final Balance
        account.displayBalance();
    }
}
```

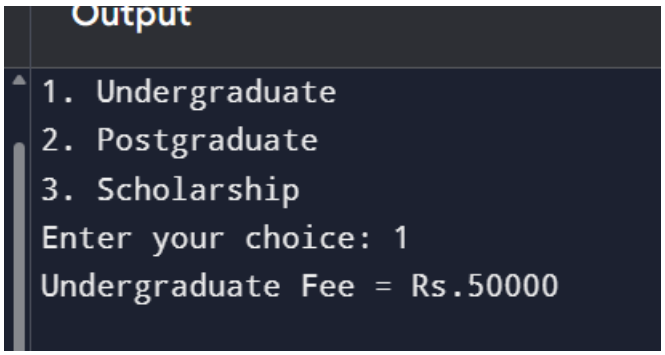
```
Amount Deposited Successfully.
Amount Withdrawn Successfully.
Insufficient Balance.
Invalid Deposit Amount.

Account Number : 1001
Account Holder : AK Selva
Available Balance : Rs.5500.0
```

5: University Admission System using Method Overloading

```
import java.util.Scanner;

class Admission {

    // Undergraduate Fee
    void calculateFee() {
        System.out.println("Undergraduate Fee = Rs.50000");
    }

    // Postgraduate Fee
    void calculateFee(int pg) {
        System.out.println("Postgraduate Fee = Rs.80000");
```

```
    }

    // Scholarship Student Fee
    void calculateFee(String scholarship) {
        System.out.println("Scholarship Student Fee = Rs.20000");
    }
}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Admission a = new Admission();

        System.out.println("1. Undergraduate");
        System.out.println("2. Postgraduate");
        System.out.println("3. Scholarship");
        System.out.print("Enter your choice: ");

        int choice = sc.nextInt();

        switch(choice) {

            case 1:
                a.calculateFee();
                break;

            case 2:
```

```
            a.calculateFee(1);
            break;

        case 3:
            a.calculateFee("Scholarship");
            break;

        default:
            System.out.println("Invalid Choice");
      }
   }
}
```

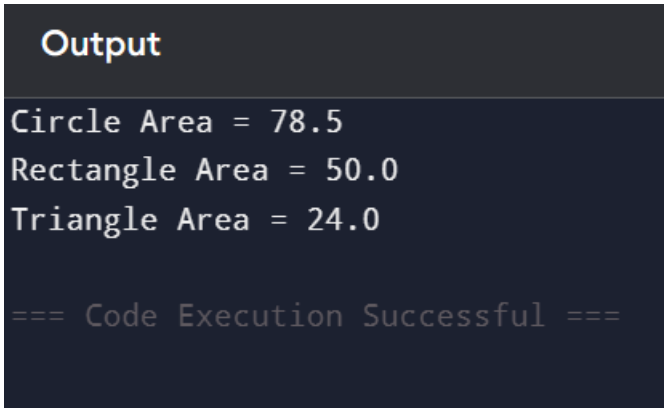


6: Shape Area Calculator using Abstract Class & Polymorphism

```
abstract class Shape {

   abstract void area();
}

class Circle extends Shape {

   double radius;
```

```
    Circle(double radius) {
        this.radius = radius;
    }

    void area() {
        System.out.println("Circle Area = " + (3.14 * radius * radius));
    }
}

class Rectangle extends Shape {

    double length, breadth;

    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }

    void area() {
        System.out.println("Rectangle Area = " + (length * breadth));
    }
}

class Triangle extends Shape {

    double base, height;

    Triangle(double base, double height) {
```

```
        this.base = base;
        this.height = height;
    }

    void area() {
        System.out.println("Triangle Area = " + (0.5 * base * height));
    }
}

public class Main {

    public static void main(String[] args) {

        Shape[] shapes = new Shape[3];

        shapes[0] = new Circle(5);
        shapes[1] = new Rectangle(10, 5);
        shapes[2] = new Triangle(8, 6);

        for (Shape s : shapes) {
            s.area();
        }
    }
}
```



7: Hospital Management System using Interfaces

```
interface MedicalRecord {

    void addRecord();

    void displayRecord();
}

class Patient implements MedicalRecord {

    String name = "Rahul";
    int patientId = 101;

    public void addRecord() {
        System.out.println("Patient Record Added Successfully");
    }

    public void displayRecord() {
        System.out.println("Patient ID : " + patientId);
        System.out.println("Patient Name : " + name);
    }
}
```

```
class Doctor implements MedicalRecord {

    String name = "Dr. Kumar";
    int doctorId = 201;

    public void addRecord() {
        System.out.println("Doctor Record Added Successfully");
    }

    public void displayRecord() {
        System.out.println("Doctor ID : " + doctorId);
        System.out.println("Doctor Name : " + name);
    }
}

public class Main {

    public static void main(String[] args) {

        MedicalRecord m;

        m = new Patient();
        m.addRecord();
        m.displayRecord();

        System.out.println();

        m = new Doctor();
```

```
        m.addRecord();
        m.displayRecord();
    }
}
```

```
Output

Patient Record Added Successfully
Patient ID : 101
Patient Name : Rahul

Doctor Record Added Successfully
Doctor ID : 201
Doctor Name : Dr. Kumar

=== Code Execution Successful ===
```

8: Online Shopping Order System

```
class Product {

    int productId;
    String productName;
    double price;

    Product(int id, String name, double price) {
        productId = id;
        productName = name;
        this.price = price;
    }

    void displayProduct() {
        System.out.println("Product ID : " + productId);
```

```
        System.out.println("Product Name : " + productName);
        System.out.println("Price : Rs." + price);
    }
}

class Order {

    Product product;
    int quantity;

    Order(Product product, int quantity) {
        this.product = product;
        this.quantity = quantity;
    }

    void generateBill() {
        double total = product.price * quantity;

        System.out.println("\n----- BILL -----");
        product.displayProduct();
        System.out.println("Quantity : " + quantity);
        System.out.println("Total Amount : Rs." + total);
    }
}

public class Main {

    public static void main(String[] args) {
```

```
        Product p = new Product(101, "Laptop", 50000);

        Order o = new Order(p, 2);

        o.generateBill();
    }
}
```

```
Output

----- BILL -----
Product ID : 101
Product Name : Laptop
Price : Rs.50000.0
Quantity : 2
Total Amount : Rs.100000.0

=== Code Execution Successful ===
```

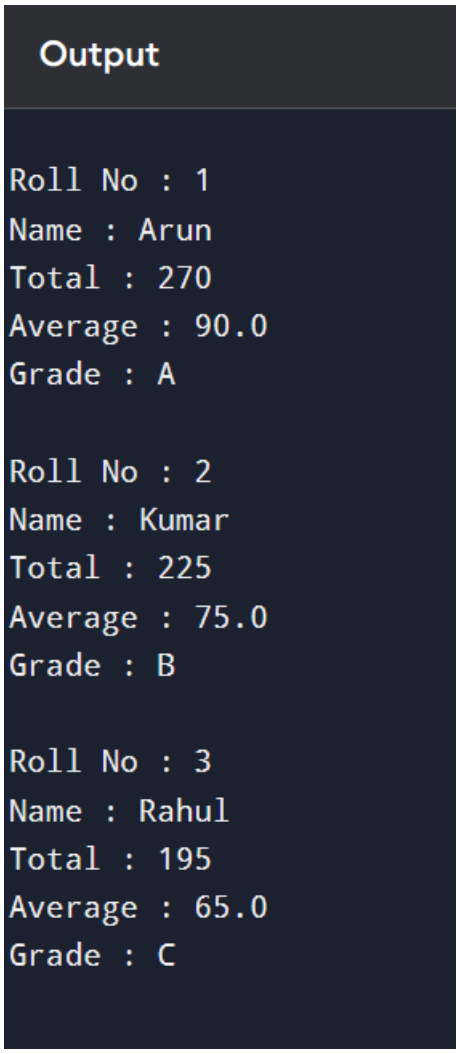
9: Student Result Analyzer using Arrays of Objects

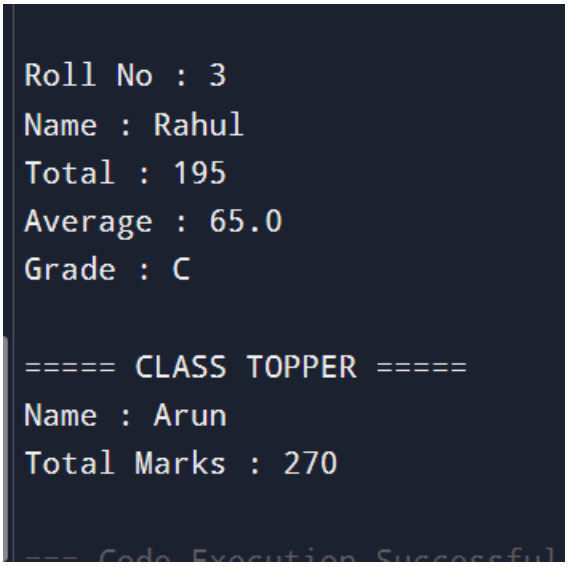
```
class Student {

    int rollNo;
    String name;
    int m1, m2, m3;

    Student(int rollNo, String name, int m1, int m2, int m3) {
        this.rollNo = rollNo;
        this.name = name;
        this.m1 = m1;
```

```
    this.m2 = m2;
    this.m3 = m3;
}

int total() {
    return m1 + m2 + m3;
}

double average() {
    return total() / 3.0;
}

String grade() {

    double avg = average();

    if (avg >= 90)
        return "A";
    else if (avg >= 75)
        return "B";
    else if (avg >= 50)
        return "C";
    else
        return "Fail";
}

void display() {
    System.out.println("\nRoll No : " + rollNo);
    System.out.println("Name : " + name);
```

```
        System.out.println("Total : " + total());
        System.out.println("Average : " + average());
        System.out.println("Grade : " + grade());
    }
}

public class Main {

    public static void main(String[] args) {

        Student students[] = new Student[3];

        students[0] = new Student(1, "Arun", 95, 90, 85);
        students[1] = new Student(2, "Kumar", 80, 75, 70);
        students[2] = new Student(3, "Rahul", 60, 65, 70);

        Student topper = students[0];

        for (Student s : students) {

            s.display();

            if (s.total() > topper.total()) {
                topper = s;
            }
        }

        System.out.println("\n===== CLASS TOPPER =====");
        System.out.println("Name : " + topper.name);
```

```
        System.out.println("Total Marks : " + topper.total());
    }
}
```





10: Smart Home Device Controller using Hierarchical Inheritance

```
class Device {

    String name;
    int power;

    Device(String name, int power) {
        this.name = name;
```

```
        this.power = power;
    }

    void turnOn() {
        System.out.println(name + " is ON");
    }

    void turnOff() {
        System.out.println(name + " is OFF");
    }

    void displayPower() {
        System.out.println("Power Consumption : " + power + " Watts");
    }
}

class Light extends Device {

    Light() {
        super("Light", 20);
    }
}

class Fan extends Device {

    Fan() {
        super("Fan", 75);
    }
}
```

```
class AirConditioner extends Device {

    AirConditioner() {
        super("Air Conditioner", 1500);
    }
}

public class Main {

    public static void main(String[] args) {

        Device devices[] = new Device[3];

        devices[0] = new Light();
        devices[1] = new Fan();
        devices[2] = new AirConditioner();

        for (Device d : devices) {

            d.turnOn();
            d.displayPower();
            d.turnOff();

            System.out.println();
        }
    }
}
```

Output

```
Light is ON
Power Consumption : 20 Watts
Light is OFF

Fan is ON
Power Consumption : 75 Watts
Fan is OFF

Air Conditioner is ON
Power Consumption : 1500 Watts
Air Conditioner is OFF
```